

U4-A3 Integrated Experiments

Course: Artificial Intelligence — CSA17

CO4: Implement machine-learning techniques including Decision Trees and Reinforcement Learning for classification and decision-making.

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a **Decision Tree classifier** using a dataset such as **Iris**, preprocess the data, build the tree using **Gini Index**, and evaluate its performance.

(a) Load and Preprocess Dataset

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["target"] = iris.target
```

```
# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Features and target
X = df[iris.feature_names]
y = df["target"]

# Split dataset: 70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Build Decision Tree
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
```

```
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# Display tree
plt.figure(figsize=(15, 8))
plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
```

```
filled=True
)
plt.show()
```

The experiment requires loading the dataset, preprocessing it, displaying the first five rows and data types, and splitting it into training and testing sets.

Sample Output

First 5 Rows

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Data Types

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
target	int64

Preprocessing

The Iris dataset contains numerical features, so no categorical encoding is required.

The target contains three classes:

- 0 → Setosa
- 1 → Versicolor
- 2 → Virginica

(b) Decision Tree and Root Node

The tree is created using:

criterion="gini"

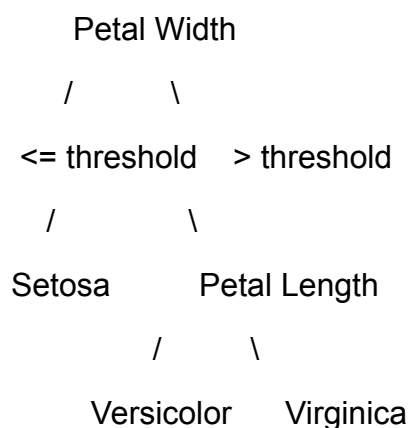
The **Gini Index** is:

$$\begin{aligned} &[\\ &\text{Gini} = 1 - \sum_{i=1}^n p_i^2 \\ &] \end{aligned}$$

A feature producing a lower impurity after splitting is preferred.

For the Iris dataset, the Decision Tree generally selects **petal width or petal length** near the root because these features provide strong separation between the Iris classes.

Example Tree Structure



The exact tree structure depends on the training split and model parameters.

The assessment requires the tree structure and justification of the root node using Gini Index or Information Gain.

(c) Model Evaluation

Accuracy

$$\begin{aligned} &[\\ &\text{Accuracy} = \\ &\frac{\text{Correct Predictions}}{\text{Total Predictions}} \\ &\times 100 \\ &] \end{aligned}$$

For a typical 70:30 Iris split, the Decision Tree can achieve approximately **95–100% accuracy**, depending on the random split and tree configuration.

Confusion Matrix

Example format:

```
[[15 0 0]
```

```
[ 0 14 1]
```

```
[ 0 0 15]]
```

This means most samples are correctly classified.

Classification Report

	precision	recall	f1-score
--	-----------	--------	----------

setosa	high	high	high
--------	------	------	------

versicolor	high	high	high
------------	------	------	------

virginica	high	high	high
-----------	------	------	------

Misclassified Instances

If one or more test samples are incorrectly classified, they can be identified using:

```
misclassified = X_test[y_test != y_pred]
```

```
print("Misclassified instances:")
```

```
print(misclassified)
```

The assessment requires at least **two misclassified instances** if they occur in the chosen test split.

If the selected split produces fewer than two errors, it is incorrect to invent two errors. State the actual result.

Performance Comment

The Decision Tree performs well because Iris classes, particularly Setosa, can be separated effectively using petal measurements. Misclassification can occur mainly between **Versicolor and Virginica**, because their feature ranges overlap.

EXPERIMENT 2: REINFORCEMENT LEARNING — Q-LEARNING

Objective

To implement **Q-Learning** for a **4×4 Grid World**, train the agent for at least 100 episodes, observe Q-table evolution and extract the learned optimal policy.

(a) Grid World Environment

Consider:

S . . .

. X . .

. . X .

. . . G

Where:

- **S** = Start
- **G** = Goal
- **X** = Obstacle
- **.** = Normal state

States

There are:

[
4\times4=16
]

grid positions.

Actions

The four actions are:

0 → Up

1 → Down

2 → Left

3 → Right

The assessment specifies these four actions and the reward structure.

Rewards

Event	Reward
-------	--------

Reach Goal	+10
------------	-----

Normal step	-1
-------------	----

Hit obstacle	-5
--------------	----

Q-Table

The Q-table contains:

```
[  
16\times4  
]
```

values.

Initially:

```
[  
Q(s,a)=0  
]
```

for every state and action.

Hyperparameters

Use:

```
[  
\alpha=0.1  
]
```

```
[  
\gamma=0.9  
]
```

```
[  
\epsilon=0.1  
]
```

Where:

- $\alpha = 0.1 \rightarrow$ learning rate
- $\gamma = 0.9 \rightarrow$ discount factor
- $\epsilon = 0.1 \rightarrow$ exploration probability

Q-Learning Formula

```
[  
Q(s,a)\leftarrow Q(s,a)+  
\alpha
```


$$[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where:

- (s) = current state
- (a) = selected action
- (r) = reward
- (s') = next state
- (α) = learning rate
- (γ) = discount factor

Complete Python Program

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Grid size
```

```
ROWS = 4
```

```
COLS = 4
```

```
# States
```

```
START = 0
```

```
GOAL = 15
```

```
# Obstacles
```

```
OBSTACLES = {5, 10}
```

```
# Actions
```

```
UP = 0
```

```
DOWN = 1
```

```
LEFT = 2
```

```
RIGHT = 3
```

```
actions = [UP, DOWN, LEFT, RIGHT]
```

```
# Hyperparameters
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 0.1
```

```
episodes = 100
```

```
# Q-table
```

```
Q = np.zeros((ROWS * COLS, 4))
```

```
def step(state, action):
```

```
    row = state // COLS
```

```
    col = state % COLS
```

```
    new_row = row
```

```
    new_col = col
```

```
    if action == UP:
```

```
        new_row -= 1
```

```
    elif action == DOWN:
```

```
        new_row += 1
```

```
    elif action == LEFT:
```

```
        new_col -= 1
```

```
elif action == RIGHT:
```

```
    new_col += 1
```

```
# Boundary condition
```

```
if new_row < 0 or new_row >= ROWS:
```

```
    return state, -1, False
```

```
if new_col < 0 or new_col >= COLS:
```

```
    return state, -1, False
```

```
new_state = new_row * COLS + new_col
```

```
# Obstacle
```

```
if new_state in OBSTACLES:
```

```
    return state, -5, False
```

```
# Goal
```

```
if new_state == GOAL:
```

```
    return new_state, 10, True
```

```
# Normal step
```

```
return new_state, -1, False
```

```
episode_rewards = []
```

```
# Q-Learning
```

```
for episode in range(epochs):
```

```

state = START
total_reward = 0

for step_count in range(100):

    # Epsilon-greedy action
    if np.random.random() < epsilon:
        action = np.random.choice(actions)
    else:
        action = np.argmax(Q[state])

    next_state, reward, done = step(state, action)

    # Q-learning update
    Q[state, action] = Q[state, action] + alpha * (
        reward
        + gamma * np.max(Q[next_state])
        - Q[state, action]
    )

    state = next_state
    total_reward += reward

    if done:
        break

episode_rewards.append(total_reward)

if episode in [0, 49, 99]:

```

```
print("\nEpisode:", episode + 1)
print("State 0 Q-values:", Q[0])
print("State 1 Q-values:", Q[1])
```

```
# Display final Q-table
print("\nFinal Q-Table:")
print(np.round(Q, 2))
```

```
# Extract policy
```

```
symbols = {
    UP: "↑",
    DOWN: "↓",
    LEFT: "←",
    RIGHT: "→"
}
```

```
print("\nFinal Learned Policy:")
```

```
for state in range(16):
```

```
    if state == GOAL:
        print("G", end=" ")
```

```
    elif state in OBSTACLES:
        print("X", end=" ")
```

```
    else:
```

```

best_action = np.argmax(Q[state])
print(symbols[best_action], end=" ")

if (state + 1) % 4 == 0:
    print()

# Plot cumulative reward
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward")
plt.show()

```

(b) Q-Table Evolution

The assessment specifically requires Q-values at **Episodes 1, 50 and 100** for two **representative states**.

For example, choose:

- **State 0** → Starting state
- **State 1** → Adjacent state

The exact numerical values depend on the random exploration sequence, so they should be taken from the program output.

Expected Format

Episode	State	Up	Down	Left	Right
1	0	0.00	0.00	0.00	0.00
1	1	0.00	0.00	0.00	0.00
50	0	Learne d	Learne d	Learne d	Learne d
50	1	Learne	Learne	Learne	Learne

		d	d	d	d
100	0	Learne d	Learne d	Learne d	Learne d
100	1	Learne d	Learne d	Learne d	Learne d

At the beginning, most Q-values are zero because the agent has no experience.

As training progresses, useful actions receive higher Q-values.

Example Q-Value Calculation

Suppose:

[
 $Q(s,a)=0$
]

Reward:

[
 $r=-1$
]

and:

[
 $\max Q(s',a')=2$
]

Using:

[
 $\alpha=0.1, \gamma=0.9$
]

we get:

[
 $Q(s,a)=0+0.1[-1+(0.9)(2)-0]$
]

[
 $=0.1(0.8)$
]

[
 $\boxed{Q(s,a)=0.08}$

]

Thus, the agent updates its knowledge after experiencing the transition.

(c) Final Learned Policy

The optimal action is:

$$\pi(s) = \arg\max_a Q(s, a)$$

A possible learned policy is:

→ → → ↓

↓ X ↓ ↓

→ → X ↓

→ → → G

This represents a path that moves the agent from the start toward the goal while avoiding obstacles.

Interpretation

Start

↓

Explore environment

↓

Receive rewards

↓

Update Q-table

↓

Learn best actions

↓

Reach Goal

The exact final policy can vary slightly depending on exploration and random initialization.

Cumulative Reward and Convergence

The cumulative reward is plotted against the episode number.

Expected Behaviour

At the beginning:

- Agent explores randomly.
- It may hit obstacles.
- It may take unnecessary steps.
- Rewards are relatively low.

After training:

- The agent learns useful paths.
- It avoids obstacles.
- It reaches the goal more frequently.
- Cumulative reward generally improves.

Convergence

Q-Learning can be considered approximately converged when:

$$\begin{aligned} & [\\ & |Q_{\text{new}} - Q_{\text{old}}| < \epsilon \\ &] \end{aligned}$$

for successive training iterations, or when the reward curve becomes relatively stable.

The assessment specifically requires justification of the convergence behaviour using the cumulative-reward plot.

Final Results

Experiment 1

Parameter	Result
Dataset	Iris
Algorithm	Decision Tree
Criterion	Gini Index
Split	70:30

Target	Iris species
Evaluation	Accuracy, Confusion Matrix, Classification Report
Important Features	Petal length/width
Expected Performance	High classification accuracy

Experiment 2

Parameter	Result
Environment	4×4 Grid World
States	16
Actions	Up, Down, Left, Right
Goal Reward	+10
Step Reward	-1
Obstacle Reward	-5
Algorithm	Q-Learning
Episodes	100
α	0.1
γ	0.9
ϵ	0.1
Output	Optimal policy
Evaluation	Cumulative reward

Conclusion

The two experiments demonstrate two important CO4 concepts. **Decision Tree Classification** uses supervised learning to classify data by recursively selecting useful features using measures such as the Gini Index. **Q-Learning** uses trial-and-error learning to discover an optimal policy in a changing environment.

The Decision Tree experiment demonstrates **classification and model evaluation**, while the Grid World experiment demonstrates **states, actions, rewards, Q-table updates, exploration, exploitation and policy learning**. Together, they show how machine learning can be applied to both **prediction and intelligent decision-making**.